

Document Retrieval and Retrieval-Augmented Generation with TinyLlama

Ida Nilsson

Uppsala University
Uppsala, Sweden

ida.linnea23@gmail.com

Yanitsa Stoykova

Uppsala University
Uppsala, Sweden

yanitsa.stoykova@gmail.com

Gina Welsh

Uppsala University
Uppsala, Sweden

gmwelsh97@gmail.com

1 Abstract

This study explores how TinyLlama, a small language model, performs in question-answering tasks using Retrieval Augmented Generation (RAG). Specifically, we investigated the difference in performance between a baseline document retrieval pipeline and a RAG retrieval pipeline using Sentence-BERT (SBERT) and TinyLlama, as well as the answer generation performance of TinyLlama with and without the prompt of a retrieved document. The results of the study show that the baseline document retrieval method performed better than the SBERT retrieval method, and that there were no major differences in performance between the answer generation experiments. In addition, the study highlighted potential limitations of using an automatic evaluation metric to measure the quality of answered generated by RAG.

2 Introduction

Retrieval-Augmented Generation (RAG) is a technique which aims to enhance the performance of generative language models by combining the parameterised knowledge of the language model with the non-parameterised knowledge of an external knowledge base (Lewis et al., 2020). In turn, language models offer potential improvement on traditional information retrieval (IR) systems by moving beyond surface-level text matching, capturing underlying semantic similarities between words, and using this information to generate answers for user queries that are specific to the IR data ecosystem (Gua et al., 2020). Recent work has been established evaluating the effects of RAG on Large Language Models (LLMs), but less work has been established evaluating its effect on Small Language Models (SLMs). Therefore, in this project, we evaluate the performance of RAG as an enhancement technique to the answer generation capabilities of TinyLlama (Zhang et al., 2024), a transformer decoder-only SLM trained on 1.1 billion parameters and 3 trillion tokens.

3 Research Questions

Our project explores two key topics: document retrieval and RAG answer generation. Specifically, we sought insight into the following questions:

1. Document retrieval: How does a RAG document retrieval pipeline with SBERT and TinyLlama compare to a simple document retrieval baseline?
2. Answer generation: How does TinyLlama’s one-shot answer generation performance compare to its zero-shot performance?

We hypothesised that the RAG document retrieval pipeline with SBERT would be better in performance than the simple lexical search baseline, and that TinyLlama’s one-shot answer generation performance would be higher in quality and reliability than its zero-shot answer generation performance.

4 Related Work

4.1 Parameterised and non-parameterised knowledge in information retrieval

Much of the newly-established research into RAG focuses on its ability to enhance the performance of LLMs. The parametric knowledge of LLMs is acquired through their training of neural network weights, which makes up the model’s generalisations of the training data and forms the foundation for the LLM’s generated responses (Gao et al., 2023). LLMs in themselves demonstrate undesirable side-effects, notably that of its tendency to generate falsehoods (hallucinations) which can undermine the quality and reliability of its generated answers. These side-effects can be particularly pronounced for knowledge-intensive information retrieval and answer generation tasks, that is, tasks that ‘humans could not reasonably be expected to perform without access to an external knowledge source’ (Lewis et al., 2020). As a result, the potential of RAG in anchoring the parameterised knowledge of language models to that of an external knowledge base is an emerging topic of interest in the field.

4.2 RAG and Large Language Models (LLMs)

Key previous works on the development and evaluation of RAG with LLMs include Lewis et al. (2020), which first introduces the concept of RAG before comparing the performance of RAG implementations using BART, a denoising autoencoder for pretraining sequence-to-sequence models, as the generator, and Dense Passage Retriever (DPR) as the retrieval mechanism. The study found that RAG models generate more specific,

diverse, and factual language than for the parametric-only baseline model for the knowledge-intensive natural language processing tasks.

Other works which test this idea with LLMs include Mallen (2023), which investigates the effects of parametric and non-parametric knowledge on the quality of generated responses with ten LLMs of the GPT-Neo, OPT, and GPT-3 families, for an open-domain set of topics with varying popularity (as measured by Wikipedia page views). The results of the study culminated into the key ideas that retrieval largely improves performance, non-parametric memories can at times mislead LLMs, and non-parametric memories are effective for generated answers on less popular topics than for more popular topics.

4.3 Small Language Models (SLMs)

Small Language Models (SLMs) are defined as lightweight versions of larger language models, typically ranging from 1 million to 10 billion parameters (Nguyen et al., 2024). There has been recent established work into measuring the learning abilities of SLMs for NLP tasks, such as (Magister et al., 2023), which explore SLM reasoning capabilities, and (Lu et al., 2025), which evaluate SLM performance on a range of tasks. TinyLlama (Zhang et al., 2024) is an example of an SLM trained on a smaller number of parameters (1.1 billion) but with a relatively larger amount of data (3 trillion tokens). TinyLlama demonstrates superior results compared to SLMs of similar parameter size for problem-solving and Chinese comprehension tasks. The performance of TinyLlama in a RAG setting, at the time of writing, is not explored yet in the established literature.

5 Method

We implemented a baseline information retrieval system which retrieves documents for a given query using a frequency-inverse document frequency (TF-IDF) vectorisation (Sparck Jones, 1972) and a cosine similarity measure. Building upon this baseline, we worked on three different text-retrieval and -generation setups:

1. Zero-Shot Generation System:

For zero-shot TinyLlama performance measurement, topics were matched to the query as strings. TinyLlama (version 0.8.0, via Ollama) was given the query topic as part of the prompt, as well as, if applicable, a context string consisting of the last 1-2 queries. Based on this query topic and context string, and with no further access to any examples or context document, a response to the query was generated.

2. One-Shot Generation System (Custom RAG):

For our one-shot TinyLlama performance measurement, we implemented a customised RAG pipeline which uses the local embedding model

Beijing Academy of Artificial Intelligence (BAAI)¹ to create vector representations of the source text and the query. The functions SimpleDirectoryReader and VectorStoreIndex from LlamaIndex² were used to process and retrieve the relevant text parts from the source text based on semantic similarity to the query. The text deemed relevant was passed to the TinyLlama model, which was set to respond, assuming the following identity: ‘You are an expert Q&A system that strictly operates in two modes when refining existing answers: Repeat/Rewrite.’ The custom prompt included the relevant chunks from the source text and the query. In some experiments, it also contained a command to strictly condition its answers on the given text. The model’s output included the document ID of the source text used for text retrieval and generation.

3. Retrieval-Augmented Generation (RAG) System:

A customised RAG pipeline was constructed with two distinct components: 1) a retrieval part to select the best match for the query, and a 2) generation step where the language model returns a generated response to the query based on the retrieved context. The retrieval was made from the database of documents constructed with the Chroma library³, which created a database from our existing dataset, used Sentence-BERT (SBERT) to embed the documents and queries to conduct a semantic search, and retrieved the top-k documents with the closest match. Chroma is integrated with packages such as Langchain⁴ and can be set to a preferred embedding function⁵. SBERT is based on BERT but created for the purpose of efficient and accurate semantic search and sentiment analysis by calculating cosine similarity between sentences, and it outperforms some of the most popular sentence embeddings transformers in both accuracy and efficiency (Reimers and Gurevych, 2019). When creating the database with Chroma optional metadata can be retrieved from the file, such as path or start index for split documents. After a database is created, it can be updated as needed, making it useful for cases where data might change over time. In the second part the retrieved document is passed to the TinyLlama model and a response generated in a similar manner as in the One Shot-system. This pipeline was largely inspired by a case study using a similar RAG structure built for handling question-answering tasks for a low-resource language, which showed improved results with fewer

¹<https://huggingface.co/BAAI/bge-small-en-v1.5>

²<https://docs.llamaindex.ai/en/stable>

³<https://www.trychroma.com/>

⁴<https://python.langchain.com/docs/introduction/>

⁵<https://docs.trychroma.com/docs/embeddings/embedding-functions>

hallucinations compared to a system without the retrieval component (Joshua, 2024).

For evaluation of the baseline and the retrieval pipelines, we used a precision score for the top 1 and 2 retrieved documents. To evaluate the quality and reliability of the generated responses, we applied BERTScore, which measures semantic similarity between the generated responses and the source text based on contextual embeddings (Zhang* et al., 2020).

6 Data

We used the Competitive Undergraduate Course Question-Answer Dataset created by authors and students at Carnegie Mellon University and the University of Pittsburgh (Smith et al., 2008). The dataset includes 150 files containing text from Wikipedia articles (690,000 words total), and a set of question-answer pairs, with at most one relevant article file linked to each question-answer pair.

Due to our limited computational resources, for the generation tasks, we worked with and perform experiments on a subset of 8 topics, one topic from each thematic group, retrieving and generating responses to 172 questions in total.

Thematic Group	Topic
Animals	Kangaroo
Countries	Liechtenstein
US Presidents	John Adams
Scientists	Blaise Pascal
Musical Instruments	Piano
Cities	London
Languages	English language
Artists	Pablo Picasso

Table 1: Subset of topics sampled for the experiments, organised by general topic and specific topic.

To build the Chroma database in the Classic RAG-System experiment, the documents were processed as a Directory using the Langchain-package⁶. The documents were also split into chunks of a maximum length of 2,000 tokens, as most documents were otherwise too large to be handled by TinyLlama.

7 Experiments

In order to examine the performance in question-answering tasks we performed several experiments by creating separate systems with different implementations of RAG using a SLM, and a retrieval baseline to compare them against.

7.1 Retrieval Baseline

For the retrieval baseline, we implemented a non-RAG pipeline which calculates TF-IDF vectors for all dataset

document articles as well as the given query for the retrieval system. The TF-IDF document and query vectors were of the same size as the corpus vocabulary, where each token in the vector had a TF-IDF score calculated. To score the documents in terms of their relevancy to the query, a cosine similarity measure was taken between the document article vectors and the query vector. The list of cosine similarity scores between the document vectors and the query vectors were sorted so that the top documents had the highest cosine similarity scores, and therefore were scored as the most relevant.

7.2 Generation Systems

7.2.1 Zero-Shot System

For the zero-shot system, most of the responses were coherent, the lack of memory caused the model to hallucinate to a large extent when generating responses to queries with no explicit context, such as ‘What information did he record in his diary?’. Therefore, we added context to the prompt, which included the current topic and the last 1 or 2 queries on that topic, based on the position of that query in the list of queries. The structure of our data made sure that the first query from each topic would always have explicit content.

In a series of further attempts to improve the generated responses, we refined and rephrased the prompt. We discovered that a more informal and descriptive prompt does not always produce a better output. The final version of the prompt that most evaluation tests were conducted with is: ‘Context: This is about topic. Previous question(s): context. New question: q’.

7.2.2 One-Shot System (Custom RAG)

We extended the zero-shot approach by adding a context document to the model. Each document was pre-processed to match a topic and transformed into a vector store using VectorStoreIndex with local embeddings. We used the local embedding model BAAI/bge-small-en-v1.5, as it was compatible with TinyLlama and small enough to not cause the program to crash due to memory overload. To read in the data, we used the function SimpleDirectoryReader. We then instantiated a dictionary of query engines, where we stored one query engine per topic. The document IDs were matched to the topics as strings.

In the next step, we created a dataset with topic-question pairs. For each topic-question pair, we retrieved the relevant context from the indexed document based on its semantic similarity to the query. Conditioned on the retrieved context document, the model generated a response using a local LLM (TinyLlama via Ollama, version 0.8.0). In one of our experiments, we created a custom template for the prompt, which included the retrieved contextual chunks, the query, and a command to form a response solely based on the provided context: ‘Given the following text, answer

⁶<https://python.langchain.com/docs/introduction/>

the question below. \n Text:{context_str}\n Question: {query_str}\n Answer:'. The output of the model consists of a generated response and the document ID of the source text related to the topic of the query. The results from these experiments are shown and described in the next section of this report.

7.2.3 Classic RAG-System

To implement a more 'classic' RAG-system we first created a database including all documents using the packages Chroma and Langchain. This database was then used to retrieve relevant documents based on a query, which were passed to and processed by the model to generate answers.

We made use of some of Chromas built-in methods used to retrieve information from the database. The method used in our system was .query(), which takes as input a string and embeds it using SBERT, then returns the top-k matches to that string from the database. The matches are the documents that most closely resemble or represent the query, calculated with a distance range ranging from 0 (perfect match) to 1 (not at all a match). Using only this method was surprisingly efficient and easy to use, and more importantly it retrieved the intended documents, proving the effectiveness of SBERT for semantic search. The second method used was .get(), which retrieves a specified part of the database item, such as 'source' from the metadata, which includes the whole path with filename at the end. The filenames of the retrieved files were later used to compare against the filenames in the Gold Question-Answer Dataset, to see whether our retrieval part of the system was accurate and working as expected.

Once the database was created and the question set of our 8 topics of interests extracted, we implemented the pre-trained TinyLlama LM in the second step to generate answers based on the retrieved texts. Here we used the Transformers library to load the TinyLlama tokeniser and model directly in the script, requiring no local downloads or server connections⁷. To use TinyLlama we simply put the query and the retrieved documents into its predefined prompt-template in order to generate a response. A simple memory was also added so that the model would remember the last retrieved document and take it into consideration as additional context when answering the next question. The memory is used so that the model can process queries using anaphoric expressions, such as 'What was his mother's name', and relate 'his' back to the previously mentioned person.

⁷<https://huggingface.co/TinyLlama/TinyLlama-1.1B-Chat-v0.6?library=transformers>

8 Results

8.1 Document retrieval: TF-IDF and Chroma with SBERT

The classic RAG system retrieved the relevant documents at an accuracy of 76% (131 out of 172), the settings for retrieving relevant documents were set to top-1 relevant docs. When testing with returning the top 2 documents instead, precision@2, in some cases the top document was not the correct one according to the gold standard but the second document was, which resulted in a 84.3% accuracy. We also noted that some articles are duplicates, so for these cases we have counted a retrieval as correct if a duplicate file was retrieved instead of the gold standard-file. To investigate which files were incorrectly retrieved, we created a confusion matrix over all retrieved files compared to the gold standard, organised by article topic. In Figure 1, we can see that for queries where the expected retrieval document would be related to the topic 'English language', for example, documents for other languages were retrieved instead.

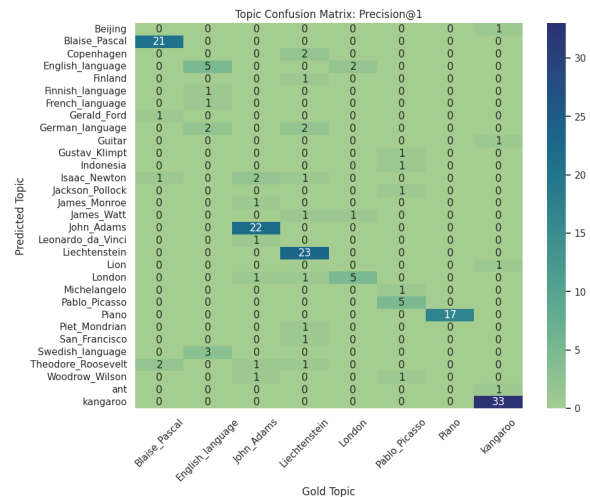


Figure 1: Precision@1 Document Retrieval with Chroma.

Retrieval	TF-IDF	Chroma
Precision@1	83.4%	76%
Precision@2	87.2%	84.3%

Table 2: Retrieval Accuracy for TF-IDF compared to Chroma.

8.2 Answer generation with TinyLlama: Zero-shot, One-shot (custom RAG), and classic RAG

We used BERTScore as the main evaluation metric to measure similarity between the generated responses and the source text. BERTScore computes a token-by-

token cosine similarity between two texts, using a pre-trained BERT model to match sentences on a semantic level rather than search for exact n-gram matches like other evaluation metrics, such as ROUGE and BLEU. BERTScore then outputs precision, recall, and F1 scores for each generated text, which we used to assess how well the meaning and the relevance of the original content have been preserved in the generated response to each query. We evaluated the generation components of our systems per response and per topic. The topic scores were calculated as the mean score of all query responses associated with a given topic. We present the results of all three answer generation pipelines in table 3.

8.2.1 Zero- and One-Shot Systems

We tested two versions of the one-shot setup: one with clearly stated instructions in the prompt for the model to condition its answers entirely on the source text, and another without explicit instructions. Table 4 summarises the BERTScore precision, recall, and F1 scores per topic for the zero-shot system, and tables 5 and 6 present the results of the experiments with the one-shot system. Overall, the BERTScore results showed minimal to no difference between these variants, suggesting that instructing the model on how to behave had a limited effect on its ability to generate the responses. The results also show that the performance of the one-shot system was comparable to that of the zero-shot system, which indicates that, at least in terms of context similarity, providing the original context to the model does not lead to any considerable improvement.

8.2.2 Classic RAG System

We evaluated each response given by our classic RAG-system in the same manner as the Zero-Shot and One-Shot, noting no major differences in the scores between the topics, indicating that the model performs equally well for all. The average F1-score is at 0.74, with a slightly lower recall (0.68) and slightly higher precision (0.81) overall, and the results for each individual topic can be seen in table 7.

	Zero-Shot	One-Shot	Classic RAG
Precision	0.836302	0.829227	0.819210
Recall	0.778355	0.780193	0.688612
F1	0.806172	0.803859	0.747260

Table 3: Results across our different generation systems: Average precision, recall, and F1 scores.

9 Discussion

We evaluated our retrieval systems according to our TF-IDF baseline. The results from the zero-shot testing were primarily used as a baseline for the generation pipelines of the RAG models and to evaluate whether or not a retrieval component, a custom prompt, and

Topic	Precision	Recall	F1
Kangaroo	0.829427	0.779867	0.803815
Liechtenstein	0.839028	0.777961	0.807242
John Adams	0.837827	0.779302	0.807378
Blaise Pascal	0.845910	0.784550	0.813946
Piano	0.826869	0.782122	0.803822
London	0.832180	0.763210	0.796178
English language	0.827927	0.780535	0.803411
Pablo Picasso	0.851256	0.779301	0.813586

Table 4: Zero-shot generation system: Precision, recall, and F1 score results per topic.

Topic	Precision	Recall	F1
Kangaroo	0.825054	0.780521	0.802115
Liechtenstein	0.835141	0.779840	0.806395
John Adams	0.828192	0.778246	0.802384
Blaise Pascal	0.838913	0.781415	0.809081
Piano	0.824180	0.780222	0.801549
London	0.833670	0.774066	0.802727
English language	0.815381	0.785971	0.800239
Pablo Picasso	0.833290	0.781264	0.806388

Table 5: One-shot generation system with explicit instructions: Precision, recall, and F1 BERT-Score results per topic.

the pre-given context improve the quality of the generated responses. While the classic RAG model focused equally on document retrieval and text generation, the custom RAG model was designed to test a vector-based semantic search approach and use it for more advanced prompt engineering. In our different approaches, we analysed the impact of each component - retrieval method, context inclusion, and prompt design - on the accuracy, relevance, and faithfulness of the generated responses with TinyLlama. To ensure that BERTScore provided an accurate representation of how good the responses are, we also performed a manual analysis with approximately 5 generated responses per topic per system.

The analysis revealed that the zero-shot generated responses were generally coherent and grammatically correct, but contained a notable number of hallucinations. Some generated texts contained misspelled words, and even relevant contextual words, for instance, words describing the main topics of the responses were often misspelled, such as ‘kanjaroo’ or ‘LieCHENSTAIN’. The one-shot responses also contained some misspellings, but they were less frequent. Moreover, the one-shot responses contained more relevant content, and in most cases, if the answer to the query was not clearly found in the original text, the model would output a response stating that: ‘The given material does not mention [...]’. In some cases, it would then still try to come up with an answer, but it would point out that it is its own ‘knowledge’. There was no distinct difference between the generated re-

Topic	Precision	Recall	F1
Kangaroo	0.827963	0.781202	0.803861
Liechtenstein	0.828431	0.779831	0.803258
John Adams	0.828408	0.778168	0.802415
Blaise Pascal	0.831822	0.779921	0.804957
Piano	0.827858	0.778741	0.802504
London	0.822728	0.765252	0.792868
English language	0.822072	0.782462	0.801698
Pablo Picasso	0.836276	0.769124	0.801224

Table 6: One-shot generation system without explicit instructions: Precision, recall, and F1 BERT-Score results per topic.

Topic	Precision	Recall	F1
Kangaroo	0.827963	0.695215	0.755006
Liechtenstein	0.824890	0.668450	0.737005
John Adams	0.816005	0.682608	0.742200
Blaise Pascal	0.803822	0.661675	0.724813
Piano	0.839966	0.722914	0.776369
London	0.799831	0.692840	0.741952
English language	0.834103	0.704323	0.763217
Pablo Picasso	0.807102	0.680870	0.737517

Table 7: Classic RAG-system: Precision, recall, and F1 score results per topic.

sponses with instructions and without.

Our key finding regarding the evaluation with BERTScore was that while it is an efficient tool for comparing machine-generated text to an original document, in many cases it would provide misleading results. Since it compares the embeddings of the generated text and its source material, a higher score can be given even to responses that a human annotator would classify as wrong for the sole reason that there is a high number of matching word representations. For instance, in our manual error analysis, we encountered errors caused by the model repeating almost the same sentence (with slight modifications) as the source text and the query, although the response starts with an explicit ‘no’ where the gold-standard answer was ‘yes’.

The BERTScore, however, catches the context similarity and ignores the negation, therefore assigning a high score to - for the human annotator - a wrong response. For example, for the query ‘*Did Bartolomeo Cristofori invent the modern piano?*’, the model responds that he did not, even though in the correctly retrieved reference document it is clearly stated that he did indeed invent the piano. This response still received a 0.78 F1-score. This example comes from the generated data of our classic RAG system and it is worth mentioning that in this case, the correct document was retrieved, so without the manual analysis, this error would have gone unnoticed. For future experiments, we think it might be beneficial to adjust the weights of the model so that it considers factual or definite state-

ments, such as ‘yes’ or ‘no’, more strongly.

For all systems, it was the case that sometimes the model would output instructions on how to formulate the prompt and not create an answer to the prompt it was given. An example of this is shown below. To save space, we only include a fraction of the generated response.

Query: What is a collective noun for kangaroos?
Response: To address your concerns about using the Repeat/Rewrite rewrite mode when operating in a two-mode system, we would recommend the following steps: [...].

This response received a BERTScore of 0.785 precision, 0.763 recall, and 0.774 F1 score, which further highlights the limitations of BERTScore as an evaluation metric when it comes to judging the relevance and faithfulness of the generated text.

10 Limitations

One important limitation of our text generation approach is the lack of multi-shot testing, which could provide the model with a better understanding of what output is expected. While we could manually adjust the model’s settings in the script or as part of the given prompts, such as the agent’s tone and role or responses’ length and style, examples of query-response pairs could have improved the overall content and coherence of the responses. Since our data corpus only includes short answers to the queries, we discussed different ways of creating our own examples to feed to the model. One idea was to use the generated responses from the one-shot tests and improve their quality by ranking them by quality and relevance. However, that would have included a rather high amount of manual, time-consuming work and introduced several additional problems, such as potential biases and inconsistencies in our gold standard responses through the manual ranking and across different evaluation metrics.

One limitation of the classic RAG system was that many documents turned out to be too large to be processed by the model. We worked around that by creating chunks of 2,000 tokens, and each chunk was connected to its original filename in the metadata. That enabled the model to process the files faster, while still being able to return the most relevant document for the query. The metadata also stated at what index a document was split, so if needed, it was possible to check what parts of the document were most relevant to the query, such as the beginning or the end. However, we could not test if it would have been more beneficial for the model to process every document in its entirety.

Since our dataset did not contain gold-standard data with a document relevancy ranking list for each question-answer pair (only at most one document was aligned as relevant to a given query) we could not measure the performance of Chroma and SBERT in ranking

several documents as potentially relevant for a given query. As a result, we were limited in our measurement of the full document retrieval capabilities of the baseline compared to the SBERT retrieval.

The last, and by far the biggest challenge, was the reliability of BERTScore as an evaluation metric. Ideally, we would have liked to perform additional tests with other methods, such as the RAGAS Faithfulness metric (Es et al., 2024), but it required a considerable amount of computational resources, which, unfortunately, we did not have at our disposal. For future works, we would aim for a more comprehensive and robust evaluation system.

11 Ethical Considerations

It is not safe to assume that the system will run without hallucinations and will always provide accurate results, as this goes against our results at this stage. While the accuracy at retrieving documents is high and the generated answers have a good BERTScore, these scores can be misleading, as previously discussed.

A big issue or even danger with using AI for answer generation is if the answer is taken as true without further thought or questioning. Often, an AI will respond with such certainty backed up ‘sources’ that it can easily fool most people. Even using a small language model that retrieved all its sources from our pre-defined database resulted in hallucinations and incorrect responses, sometimes very difficult to catch unless manually sifting through all responses and resources. While statistics can provide great insights on how a model is performing on a larger scale, human interpretations are still vital for close-up error analysis.

12 Author Contributions

In this project, we divided the work up as equally as we could, partly because we were all interested in learning more about RAG and the technical aspects of implementing different approaches. While helping each other along the way and working together, we did divide some responsibilities to work more efficiently. Gina created our TF-IDF baseline and conducted most of the background research, Yanitsa our Zero- and One-Shot experiments and Ida the classic RAG-pipeline. As such, we wrote the corresponding parts in the report, but also proofread the others. The remaining parts of the report were written together and parts cannot be attributed to an individual author.

All code and results for this project can be found in its GitHub repository.⁸

13 AI Contributions

In our group we all have a very similar view on how we want to limit the use of AI and chat bots as much as

possible, especially when it comes to academic work, so that we can proudly present our work as our own.

For transparency, we did use ChatGPT to get help with understanding some error messages, in particular issues with implementing Langchain and TinyLlama storage handling.

14 Conclusions

In conclusion, we did not find any notable differences in performance between any of our implementations. Our baseline performed better at retrieving documents than our Chroma retriever did, and the Zero- and One-Shot systems generated slightly better answers than the classic RAG, when comparing the BERTScores for each system. For future research, we would make use of a different method for evaluating the generated answers and investigate if it is in fact the small-sized language model that causes the hallucinations by testing with a slightly larger model.

References

- [Es et al.2024] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. RAGAS: Automated evaluation of retrieval augmented generation. In Nikolaos Aletras and Orpheus De Clercq, editors, *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, St. Julians, Malta, March. Association for Computational Linguistics.
- [Gao et al.2023] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1).
- [Guu et al.2020] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. 2020. Retrieval augmented language model pre-training. In *International conference on machine learning*, pages 3929–3938. PMLR.
- [Joshua2024] Adejumbi Monjolaoluwa Joshua. 2024. Improving question-answering capabilities in large language models using retrieval augmented generation (RAG): A case study on yoruba culture and language. In *5th Workshop on African Natural Language Processing*.
- [Lewis et al.2020] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474.
- [Lu et al.2025] Zhenyan Lu, Xiang Li, Dongqi Cai, Rongjie Yi, Fangming Liu, Xiwen Zhang,

⁸<https://github.com/ginawelsh/RAGTinyLlama>

Nicholas D. Lane, and Mengwei Xu. 2025. Small language models: Survey, measurements, and insights.

[Magister et al.2023] Lucie Charlotte Magister, Jonathan Mallinson, Jakub Adamek, Eric Malmi, and Aliaksei Severyn. 2023. Teaching small language models to reason.

[Nguyen et al.2024] Chien Van Nguyen, Xuan Shen, Ryan Aponte, Yu Xia, Samyadeep Basu, Zhengmian Hu, Jian Chen, Mihir Parmar, Sasidhar Kunapuli, Joe Barrow, Junda Wu, Ashish Singh, Yu Wang, Jixiang Gu, Franck Dernoncourt, Nesreen K. Ahmed, Nedim Lipka, Ruiyi Zhang, Xiang Chen, Tong Yu, Sungchul Kim, Hanieh Deilamsalehy, Namyong Park, Mike Rimer, Zhehao Zhang, Huanrui Yang, Ryan A. Rossi, and Thien Huu Nguyen. 2024. A survey of small language models.

[Reimers and Gurevych2019] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 11.

[Smith et al.2008] Noah Smith, Michael Heilman, and Rebecca Hwa. 2008. Question generation as a competitive undergraduate course project. 01.

[Sparck Jones1972] Karen Sparck Jones. 1972. A statistical interpretation of term specificity and its application in retrieval. *Journal of documentation*, 28(1):11–21.

[Zhang* et al.2020] Tianyi Zhang*, Varsha Kishore*, Felix Wu*, Kilian Q. Weinberger, and Yoav Artzi. 2020. Bertscore: Evaluating text generation with bert. In *International Conference on Learning Representations*.

[Zhang et al.2024] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. Tinyllama: An open-source small language model.